

1.a.i.

Giving the ICUTypes increasing values might tell the model the ICUTypes are ordered or spaced out numerically which will cause bias in the system. A better alternative would be to have each ICUType be a separate variable and have each be either True or False (or 0 or 1 for binary representation).

1.a.ii.

Because we only consider the maximum value, we cannot capture how the patients' condition has changed throughout their visit. Additionally, because we only take the maximum, we have no visibility into low numbers, which could be equally as concerning (i.e. if a patients' temperature dropped significantly). To capture more information, taking the minimum, maximum, and mean would be more helpful.

1.b

This imputation approach assumes that values are missing at random. Another approach could be using the median instead in order to capture more of a trend in the data. However, computing the median is far less efficient.

1.c.

Without feature normalization regularization unproportionally penalizes features on smaller scales and shrinks their influence relative to features on larger scales. With normalization, we ensure all features are on a "level playing field" and allow weights to be penalized more precisely.

1.d

Feature	Mean Value	Interquartile Range
Age	0.6471	0.3378
Gender	0.5790	1.0000
Height	0.3722	0.0008
ICUType	0.5933	0.6667
Weight	0.2950	0.1136
max_ALP	0.0717	0.0165
max_ALT	0.0156	0.0137
max_AST	0.0157	0.0138
max_Albumin	0.5053	0.0000
max_BUN	0.1319	0.0979
max_Bilirubin	0.0430	0.0252
max_Cholesterol	0.3261	0.0000
max_Creatinine	0.0649	0.0334
max_DiasABP	0.3330	0.0394
max_FiO2	0.7504	0.4167
max_GCS	0.8772	0.2500
max_Glucose	0.1389	0.0684
max_HCO3	0.3503	0.1389
max_HCT	0.3335	0.1459
max_HR	0.3294	0.1835
max_K	0.2094	0.1014
max_Lactate	0.1360	0.0498
max_MAP	0.3996	0.0582
max_Mg	0.0415	0.0143
max_NIDiasABP	0.2995	0.1149
max_NIMAP	0.3394	0.1081
max_NISysABP	0.5512	0.1087
max_Na	0.3550	0.0820
max_PaCO2	0.3362	0.1026
max_PaO2	0.4671	0.3232
max_Platelets	0.2016	0.1113
max_RespRate	0.1719	0.0000
max_SaO2	0.9419	0.0092
max_SysABP	0.5436	0.0644
max_Temp	0.3609	0.1538
max_TroponinI	0.1938	0.0000
max_TroponinT	0.0579	0.0000
max_Urine	0.0781	0.0562
max_WBC	0.0018	0.0006
max_pH	0.0048	0.0015

2.a.

It might be beneficial to maintain class proportions across folds so that each once reflects the true distribution of positive and negative examples. Without consistent proportions, we might have unstable metrics or could lead to some misleading results.

2.b.

C is the inverse of the regularization strength. When we have a small C, we have high bias. In contrast, when we have a large C, we have high variance. So, C makes our model more complex.

2.c.

Performance Measure	C	Penalty	Mean (Min, Max) CV Performance
accuracy	0.100	L1	0.8606 (0.8594, 0.8625)
precision	1.000	L2	0.6024 (0.1667, 1.0000)
f1-score	0.001	L1	0.2456 (0.2418, 0.2466)
auroc	1.000	L2	0.7804 (0.7539, 0.8192)
average_precision	1.000	L2	0.3923 (0.2893, 0.4965)
sensitivity	0.001	L1	1.0000 (1.0000, 1.0000)
specificity	0.001	L2	1.0000 (1.0000, 1.0000)

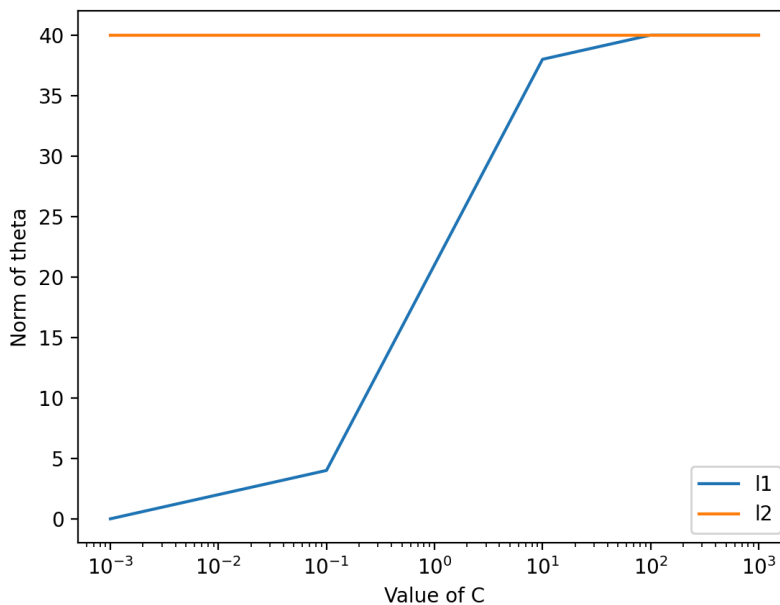
At very small values of C, our model is very biased which makes it base predictions on a single class. This explains why in the results sensitivity and specificity reached 1.0 while other metrics with the same parameters dropped close to zero (not necessarily shown in the table). As C reaches the "medium" values, bias weakens in the training, and the model starts to fit the data a little more as seen in the auroc and accuracy scores. As C increases to 10, 100 and onwards, the model begins to have high variance and starts to overfit the data.

I would optimize the AUROC because it evaluates performance across all possible thresholds and makes it more robust to class imbalance. Accuracy is definitely still a good measure to check in on, but the AUROC optimization will ensure we are generalizing well to new data.

2.d.

Performance Measure	Median	95% Confidence Interval
accuracy	0.8575	(0.8200, 0.8875)
precision	0.4167	(0.1429, 0.7273)
f1_score	0.1449	(0.0351, 0.2623)
auroc	0.7832	(0.7272, 0.8348)
average_precision	0.3586	(0.2472, 0.4760)
sensitivity	0.0877	(0.0200, 0.1703)
specificity	0.9798	(0.9645, 0.9942)

2.e.



In the plot, at small values of C nearly all weights are zero, and as C increases more features become active until most coefficients are nonzero. In contrast, the L2 penalty produces a nearly constant L0-norm, since it shrinks weights but rarely drives them exactly to zero.

2.f.

Positive Coefficient Feature Name

4.1053	max_Bilirubin
2.5931	max_BUN
1.6193	max_HR
1.5566	Age

Negative Coefficient Feature Name

-2.7041	max_GCS
-2.0179	max_Urine
-0.9237	max_SaO2
-0.8589	max_HCO3

2.g.

In simple terms, if a coefficient's term is more positive the the model believes that the category contributes more towards +1 (patient death), if it is more negative the model believe it contributes more towards -1 (patient survival) and if it is closer to 0 the model believes it has little effect on the prediction.

3.1.

3.1. for $y=+1, y'=1$

$$\left. \begin{aligned} \text{loss}_{ce}(\bar{x}, y, \bar{\theta}) &= -y' \ln \sigma(\bar{\theta} \cdot \bar{x}) - (1-y') \ln(1 - \sigma(\bar{\theta} \cdot \bar{x})) \\ &= -\ln(\sigma(\bar{\theta} \cdot \bar{x})) \end{aligned} \right\} \begin{aligned} \text{loss}_l(\bar{x}, y, \bar{\theta}) &= \ln(1 + e^{-y \bar{\theta} \cdot \bar{x}}) \\ &= \ln(1 + e^{-\bar{\theta} \cdot \bar{x}}) \end{aligned}$$

for $y=-1, y'=0$

$$\text{loss}_{ce} = -\ln(1 - \sigma(\bar{\theta} \cdot \bar{x})) \quad \text{loss}_l = \ln(1 + e^{\bar{\theta} \cdot \bar{x}})$$

→ the two losses are identical for $y=+1, y'=1$ and $y=-1, y'=0$

3.2.a.

If W_p is a lot greater than W_n it means the model places much greater emphasis on correctly classifying positive cases and heavily penalizes false negatives. Under the hood, this shifts the decision boundary toward predicting more positives and will increase sensitivity and reduce specificity. Overall, the model will become more cautious about missing positive cases.

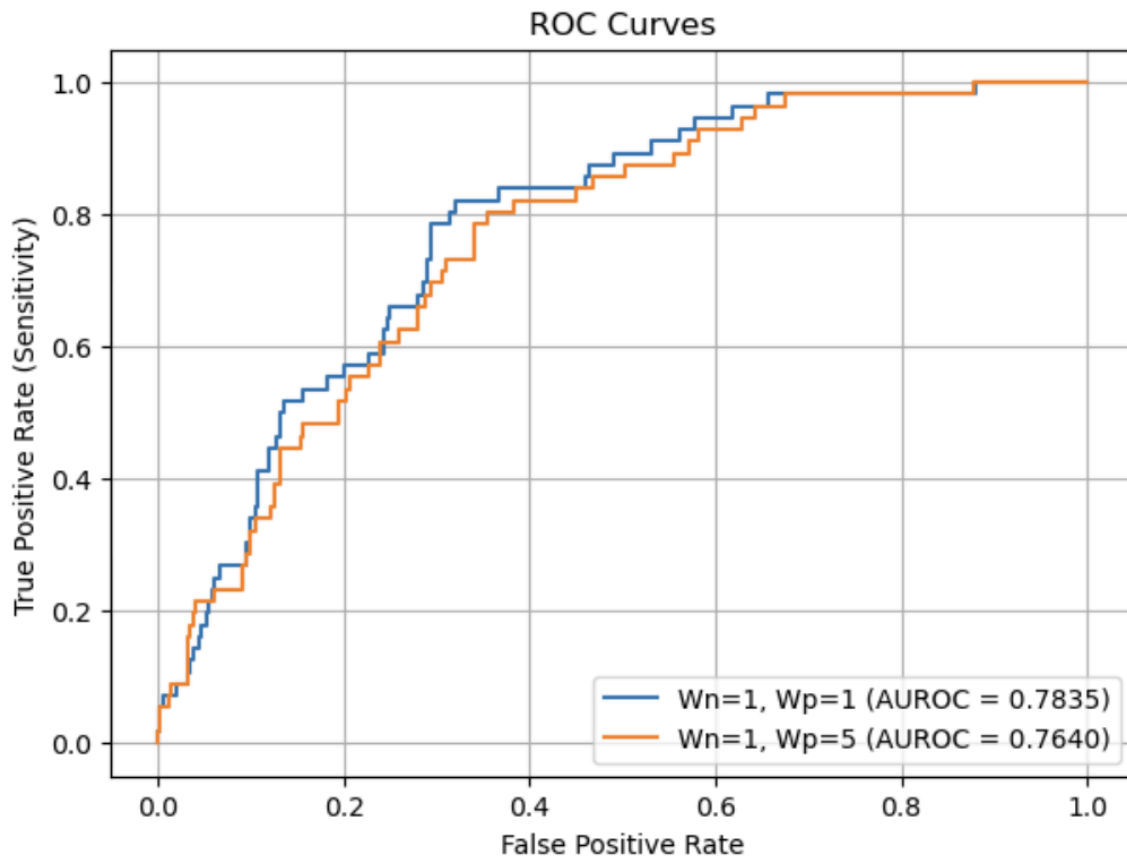
3.2.b.

Performance Measure	Median	95% Confidence Interval
accuracy	0.2400	(0.2024, 0.2850)
precision	0.1542	(0.1181, 0.1931)
f1_score	0.2671	(0.2112, 0.3237)
auroc	0.7406	(0.6781, 0.7977)
average_precision	0.3493	(0.2367, 0.4605)
sensitivity	1.0000	(1.0000, 1.0000)
specificity	0.1192	(0.0850, 0.1557)

3.2.c.

Sensitivity and Specificity are the two categories that are most drastically changed. Sensitivity increases because the model is penalized for false negatives and is incentivized to predict positives when it maybe wouldn't have without the weights. On the flip side, the specificity decreases significantly because we have more false negatives.

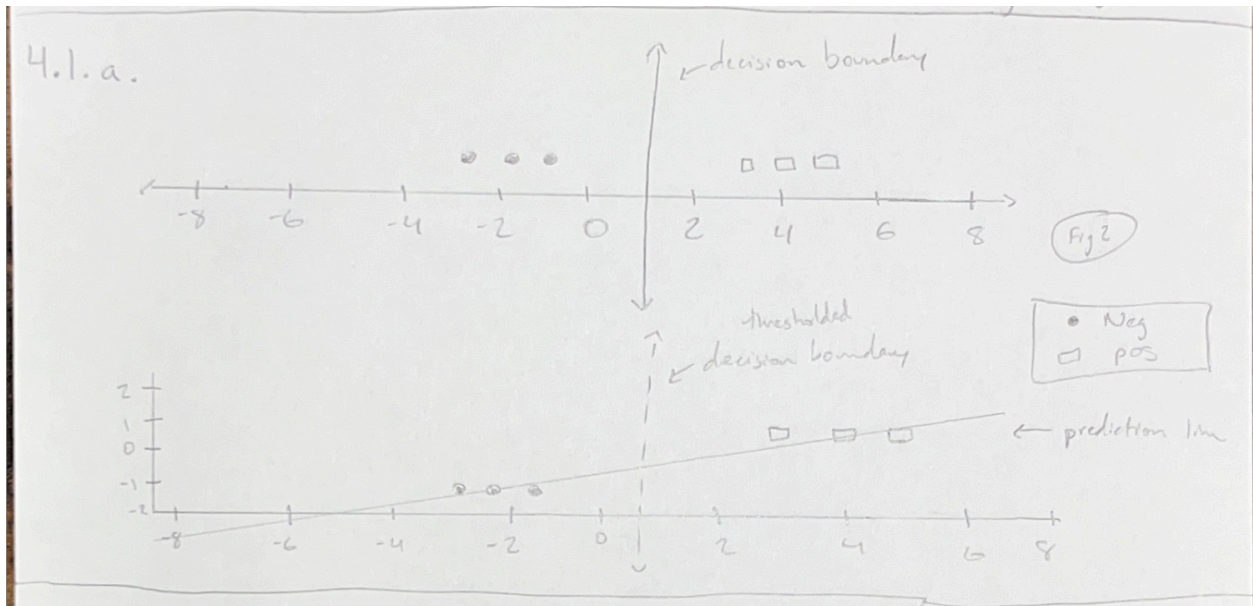
3.3.a.



3.3.b.

One method would be to decrease the decision threshold when classifying data points. Typically, if a predicted probability is > 0.5 we label it as +1, but if the model is already fit on imbalanced data, we could lower the threshold to something like > 0.25 and limit the number of false negatives.

4.1.a



4.1.b.

----- Logistic -----		
Metric	Median	95% Confidence Interval
accuracy	0.8575	(0.8200, 0.8875)
precision	0.4167	(0.1429, 0.7273)
f1_score	0.1449	(0.0351, 0.2623)
auroc	0.7829	(0.7276, 0.8351)
average_precision	0.3591	(0.2483, 0.4765)
sensitivity	0.0877	(0.0200, 0.1703)
specificity	0.9798	(0.9645, 0.9942)

----- Ridge -----		
Metric	Median	95% Confidence Interval
accuracy	0.8700	(0.8350, 0.9000)
precision	0.7303	(0.3333, 1.0000)
f1_score	0.1562	(0.0384, 0.2857)
auroc	0.7600	(0.6985, 0.8181)
average_precision	0.3463	(0.2361, 0.4541)
sensitivity	0.0877	(0.0200, 0.1703)
specificity	0.9943	(0.9853, 1.0000)

There doesn't appear to be a large difference in performance between logistic regression and kernel ridge. Both models achieve good accuracy and specificity scores but very low sensitivity. This is likely due to the class imbalance in the data. Both are linear classifiers so their decision boundaries are likely about the same. The only noticeable differences are minimal differences in metrics like precision and AUROC because of the different loss functions.

4.2.b.

Gamma	Mean	(Min, Max) CV Performance
0.001	0.7467	(0.6949, 0.8029)
0.010	0.7748	(0.7405, 0.8089)
0.100	0.7915	(0.7589, 0.8248)
1.000	0.7675	(0.7253, 0.7854)
10.000	0.7485	(0.7194, 0.7840)
100.000	0.7164	(0.6744, 0.7605)

Cross-validation AUROC performance is best at gamma = 0.1 with a mean CV performance of 0.7915. For very small gammas like 0.001, the kernel is overly smooth and underfits while for a large gamma like 100, the kernel is too complex and overfits. Both of which lead to poor results. The sweet spot is the intermediate gamma values that strike the best balance between bias and variance and yield the strongest generalization.

4.2.c.

Best C: 1.0, Best Gamma: 0.1

Metric	Median	95% Confidence Interval
accuracy	0.8700	(0.8350, 0.9000)
precision	0.7303	(0.3333, 1.0000)
f1_score	0.1562	(0.0384, 0.2857)
auroc	0.7842	(0.7252, 0.8387)
average_precision	0.3676	(0.2530, 0.4821)
sensitivity	0.0877	(0.0200, 0.1703)
specificity	0.9943	(0.9853, 1.0000)

4.2.d

Because the RBF kernel essentially corresponds to an infinite-dimensional implicit feature map, there's no finite coefficient vector by feature. With logistical regression we can call `.coef_` to give us the weights by input feature (as we saw in question 2.f.), but this is not possible with KernelRidge RBF.

Challenge write up:

In the challenge section, I improved model performance by expanding the feature set beyond maximum values. Instead of using only the max for each variable, I extracted a richer set of summary statistics—minimum, mean, standard deviation, and last observed value. To further capture patient trajectory, I computed these summaries both over the full 48-hour period and within two clinically motivated windows: 0 to 24 hours and 24 to 48 hours after admission. This approach allowed the classifier to account for both baseline levels and short-term changes, which are often crucial signals in ICU outcomes. For example, the difference between first-day and second-day vitals can highlight deterioration that a single maximum would miss. This windowed feature engineering produced a more informative representation and led to stronger separation between patients who survived and those who did not.

For hyperparameter tuning, I built a systematic grid search with cross-validation to evaluate penalties (L1, L2, and elastic-net) with their compatible solvers, a wide range of C values, and multiple class-weighting strategies. I optimized the parameter search on both F1 score and AUROC. The final configuration was L1 penalty with C=10, solver=liblinear, and class weights of WN=1 and WP=5.

I especially valued applying GridSearchCV here, since I had prior exposure to it during my internship with Lumen Technologies. That experience made me more comfortable setting up multiple scoring metrics, nested parameter grids, and stratified folds. In this project, it allowed me to confirm that my final choice was not arbitrary but the result of systematic search across feasible configurations according to the sklearn documentation. Overall, the combination of trajectory-aware feature engineering and rigorous hyperparameter tuning produced a well-regularized, interpretable model suited for the challenge dataset.

